Full-Stack Engineering Training — Day 10 Study Material

MODULE	TOPIC	STACK
Advanced State Architecture	TanStack React Query + Zustand	React Query v5 • Zustand v5 • TypeScript

Key Learning Objectives: Understand the critical separation between *Server State* (remote, asynchronous, cacheable) and *Client State* (local, synchronous UI), master `useQuery` automatic caching and deduplication, implement optimistic mutations with rollback capabilities, and construct lightweight persistent stores using `Zustand`.

1. Mental Model: Server State vs Client State

Attempting to store remote server responses in traditional global stores like Redux creates massive synchronization headaches. Modern enterprise architectures separate state into two distinct categories:

Dimension	Server State (TanStack React Query)	Client State (Zustand)
Ownership	Remote server / database (shared across users)	Local browser session only
Nature	Asynchronous, potentially stale, requires fetch	Synchronous, instant in-memory read/write
Examples	Project list, tower status, user profiles, reports	Sidebar open/close, active filter tab, auth token
Key Challenges	Caching, deduplication, retry logic, background sync	Minimal boilerplate, selective re-renders

2. TanStack React Query Fundamentals

React Query automatically caches API responses by unique query keys and eliminates duplicate network requests across components:

```
import { useQuery } from '@tanstack/react-query';

export function useProjects() {
  return useQuery({
    queryKey: ['projects'], // Unique cache identity
    queryFn: async () => {
      const res = await fetch('/api/projects');
      if (!res.ok) throw new Error('Failed to load projects');
      return res.json();
    },
    staleTime: 1000 * 60 * 5, // Fresh for 5 minutes (no background refetch during this time)
  });
}
```

Automatic Deduplication: If 5 different components request `['projects']` simultaneously, React Query initiates only ONE HTTP request and broadcasts the result to all 5 components.

3. Optimistic UI Updates with useMutation

To provide a fast, app-like experience on slow network connections, we apply updates to the local UI cache immediately, rolling back only if the server returns an error:

```
const createMutation = useMutation({
  mutationFn: (newProject) => fetch('/api/projects', {
    method: 'POST',
    body: JSON.stringify(newProject)
  }),

  // 1. onMutate: Runs immediately BEFORE the HTTP request is sent
  onMutate: async (newProject) => {
    await queryClient.cancelQueries({ queryKey: ['projects'] });
    const previousProjects = queryClient.getQueryData(['projects']);

    // Optimistically push into local cache with temporary ID
    queryClient.setQueryData(['projects'], (old = []) => [
      ...old,
      { ...newProject, id: 'temp-' + Date.now() }
    ]);

    return { previousProjects }; // Stored in context for rollback
  },

  // 2. onError: Triggered if server rejects or network drops
  onError: (err, newProject, context) => {
    queryClient.setQueryData(['projects'], context.previousProjects);
  },

  // 3. onSettled: Triggered after success or error to sync real database state
  onSettled: () => {
    queryClient.invalidateQueries({ queryKey: ['projects'] });
  }
});
```

4. Zustand: Lightweight Client State Management

Zustand provides a direct, hook-based store without the ceremony of actions, reducers, or context wrappers:

```
import { create } from 'zustand';
import { persist } from 'zustand/middleware';

interface UIState {
  sidebarOpen: boolean;
  activeFilter: string;
  toggleSidebar: () => void;
  setFilter: (filter: string) => void;
}

export const useUIStore = create<UIState>()(
  persist(
    (set) => ({
      sidebarOpen: true,
      activeFilter: 'ALL',
      toggleSidebar: () => set((state) => ({ sidebarOpen: !state.sidebarOpen })),
      setFilter: (filter) => set({ activeFilter: filter }),
    }),
    { name: 'app-ui-preferences' } // Persists in browser localStorage
  )
);
```

5. Common Beginner Pitfalls & "Intern Watch-Outs"

> **1. Direct Cache Mutation:**
> Never write `data.push(newItem)` on React Query results! Always use `queryClient.setQueryData()` to maintain immutability.

> **2. Neglecting Selectors in Zustand:**
> Writing `const store = useUIStore()` causes your component to re-render when ANY property in the store changes. Always use granular selectors: `const sidebarOpen = useUIStore(s => s.sidebarOpen)` .

> **3. Missing Query Key Variables:**
> When fetching a single project by ID, use `['project', id]` instead of a generic key like `['project']` .

6. Summary & Memory Cheat Sheet

- **Golden Rule**: Server state → **React Query**. UI state → **Zustand**.
- **Optimistic M-E-S**: **M**utate locally → **E**rror rollback → **S**ettle & invalidate.
- **Zustand Advantage**: Zero providers, fully typed, direct selector hooks.

7. Verification & Exercise Output (o/p)

```
--- RUNNING DAY 10 STATE MANAGEMENT VERIFICATION ---
✓ QueryClient initialized with default cache configuration
✓ useProjects() fetched and cached initial dataset
✓ Optimistic mutation updated UI instantly
✓ Rollback snapshot successfully restored on simulated failure
✓ Zustand persistent UI store verified with localStorage sync
ALL DAY 10 EXERCISES VERIFIED SUCCESSFULLY!
```